ROUND 3 — THE FINAL PRODUCT

Baton — Accountable Engineering Handover Platform

Every claim carries its evidence. Baton replaces vanishing tribal knowledge with an auditable handover workspace: decisions with rationale, unfinished work with owners, contradictions settled by humans, and quotes verified by deterministic character offsets — no LLM anywhere in the evidence path.

Team GIT LOST — Aditya Verma (Team Leader) · Manas Kapoor

CodersInPune 2026 · Final Round
gitlostbaton.xyz · github.com/26f2000861-AV/baton

SECTION 01

What Changed Since Round 2: The Three Promised Improvements

In the Round 2 submission form we committed to exactly three improvements, scoped as product outcomes rather than enterprise promises. This page is the contract: what was promised, what shipped, and how a judge can verify each item in the deployed product without anyone from the team explaining anything. All three are live on **gitlostbaton.xyz** today.

Improvement	Shipped in Round 3	How to verify it
1. Persistent, authenticated workspace Real sign-in, server-side role authorization, persistent storage; a successor returns later and sees the same state; persona-switching no longer grants access.	NextAuth v4 (GitHub OAuth + allowlisted one-click demo seats), PostgreSQL on Neon via Prisma v6, multi-tenant handovers with transactional template cloning, and layered server-side guards. Browser localStorage is fully removed.	Sign in on two devices; the workspace state is identical. Call the settlement endpoint as a successor: the server answers **403 Forbidden**. A non-member handover id answers **404**, never 403, so ids cannot be enumerated.
2. Evidence-backed source import Scoped GitHub import of pull requests and commits, mapped to claims with verifiable excerpts and links back to origin, with clear import errors and unsupported sources declared.	A read-only GitHub REST v3 import engine. Merged PRs become claims with verbatim excerpts; open PRs become unfinished work items. A three-tier auth chain (user OAuth, server PAT, public) with per-job tier tracking, failure states, and friendly rate-limit errors.	Open the Sources panel and import **26f2000861-AV/baton** (or any repo). Inspect the job card, click an origin link back to GitHub, and check the character-offset quote match on an imported claim.
3. Shared, verifiable work history A shared, dated team history: entries persist with author identity, link to imported PRs, and show machine verification separately from author explanations.	Every work update records its author from the authenticated session, with rationale, next action, and linked source. The deterministic verifier renders as a separate machine-check badge, and an activity timeline carries settlements, imports, and updates across devices.	Capture a work update in one browser, then sign in as another member elsewhere: the entry, its author block, linked quote, and audit status all appear in the shared History view.

Sections 4-6 give each improvement a full page with the architecture behind it and a screenshot of the shipped behaviour. Nothing on this page is a mockup; every claim is reproducible against the production deployment.

SECTION 02

The Problem and the Users

When an engineer leaves a team, their knowledge leaves with them. What remains is scattered across closed pull requests, muted Slack channels, and the memories of people who are now busy elsewhere. The incoming successor inherits a service they must operate on day one, with no map of which decisions were deliberate, which work was abandoned mid-flight, or which "temporary" workaround has quietly become load-bearing.

The cost is not documentation time; it is risk. During the first incident after a departure, the team discovers that nobody knows who owns the payments queue, why the message broker was rolled back, or which database migration was never finished. The successor spends weeks chasing humans for answers that were written down once, somewhere, in a form nobody can find. Baton turns that chase into an inspectable ledger.

> *"The decisions already taken, what is still unfinished, who owned what, and where the risk sits - without having to chase anyone."*

Baton is built around the three seats of every real handover:

Priya Sharma - the successor (senior backend engineer) adopts unfinished work, asks targeted questions, and signs off section by section with undo. The demo seat demo_successor is the recommended judge entry point.

Alex Chen - the leaver (staff platform engineer) completes a gap-driven exit interview that upgrades claims to stated-by-leaver status with full audit timestamps, attesting knowledge while still present.

Vikram Malhotra - the lead (engineering director) settles cross-source contradictions with authoritative decision notes, manages membership, and imports real GitHub repositories as evidence.

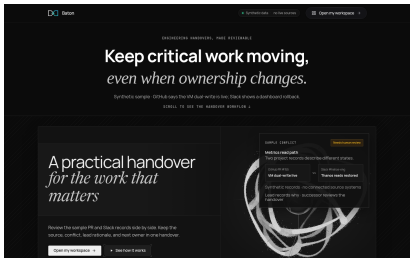
All three seats are pre-provisioned on the sign-in page as one-click, passwordless demo accounts, so an evaluator can occupy any role in the workflow without granting OAuth permissions. Each demo seat resolves through a reserved @gitlostbaton.xyz e-mail namespace, which cannot collide with a real GitHub account.

SECTION 03

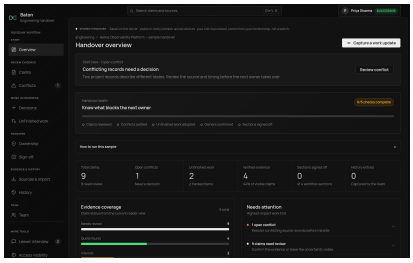
The Product Workflow

The evaluated journey runs from first sign-in to successor sign-off. Every step writes to the server database, so the state a judge creates survives refreshes, device switches, and repeat visits.

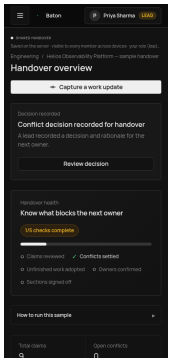
1. **Sign in and take a seat.** GitHub OAuth for real identities, or a one-click demo seat. The role is assigned server-side from the Membership row - the client is untrusted.
2. **Review clearance-filtered claims.** Each claim shows its verification status and provenance. A claim restricted to leads is withheld from the successor with a stated reason, not hidden silently.
3. **Capture decisions while working.** Engineers record what changed, why, and the next action, with an optional source reference. Entries are timestamped and attributed to the session user.
4. **Settle contradictions.** Conflicting records - a commit that says SQS versus a chat that says RabbitMQ - queue for the lead, who records a decision note or a dual-scope resolution.
5. **Import real evidence.** The lead or successor imports any GitHub repository; merged PRs become claims with verbatim excerpts and origin links, and open PRs become triage work items.
6. **Interview the leaver.** A five-question interview generated from undocumented gaps upgrades claim statuses with audit timestamps before the engineer walks out.
7. **Sign off, with an escape hatch.** The successor accepts sections one by one, can undo any acceptance, and can ask claim-linked questions that the lead answers in the same thread.



Landing page



One-click demo seats



Mobile, 390 px

Figure 1 - The entry surface on production: landing, seat picker, and mobile layout.

The workflow is built to hold up without its authors. Bad input is rejected at the route with a stated reason - an empty question, an over-long reply, a malformed repository name each return a 400 with a human-readable message instead of a silent failure. Every list view has a designed empty state ("Nothing has happened on this handover yet", "No sources yet - import a repository above"), so a fresh handover explains itself rather than rendering a blank frame. Refreshing mid-journey re-hydrates from the server, repeating an action converges on the same state, and a handover with no lead still lets its creator claim the governance seat through the founder rule - so no sequence of clicks can dead-lock the review.

SECTION 04

Improvement 1: Persistent, Authenticated Workspace

Round 2 stored every workflow change in browser localStorage: honest for a demo, useless for a handover product. Round 3 replaced it with server-side persistence and real authentication. NextAuth v4 issues JWT sessions from two providers: GitHub OAuth for real identities, and an allowlisted credentials provider whose six demo logins resolve through the reserved @gitlostbaton.xyz e-mail namespace. Because the namespace is reserved, a demo sign-in can only ever match an account the demo provider created itself - typing a real GitHub username is rejected with 401, a takeover probe we re-ran against production for this dossier.

Authorization is enforced in three layers on every route: requireSession validates the JWT (401 unauthenticated), requireMembership confirms a Membership row for that exact handover (404 for non-members, so valid handover ids cannot be enumerated), and requireRole checks the operation against the role stored in the database (403 for, say, a successor calling the lead-only settlement endpoint). Role badges and disabled buttons in the UI are conveniences only; the server is the sole authority. The persona switcher from Round 2 is gone - a seat is who you are, not a dropdown.

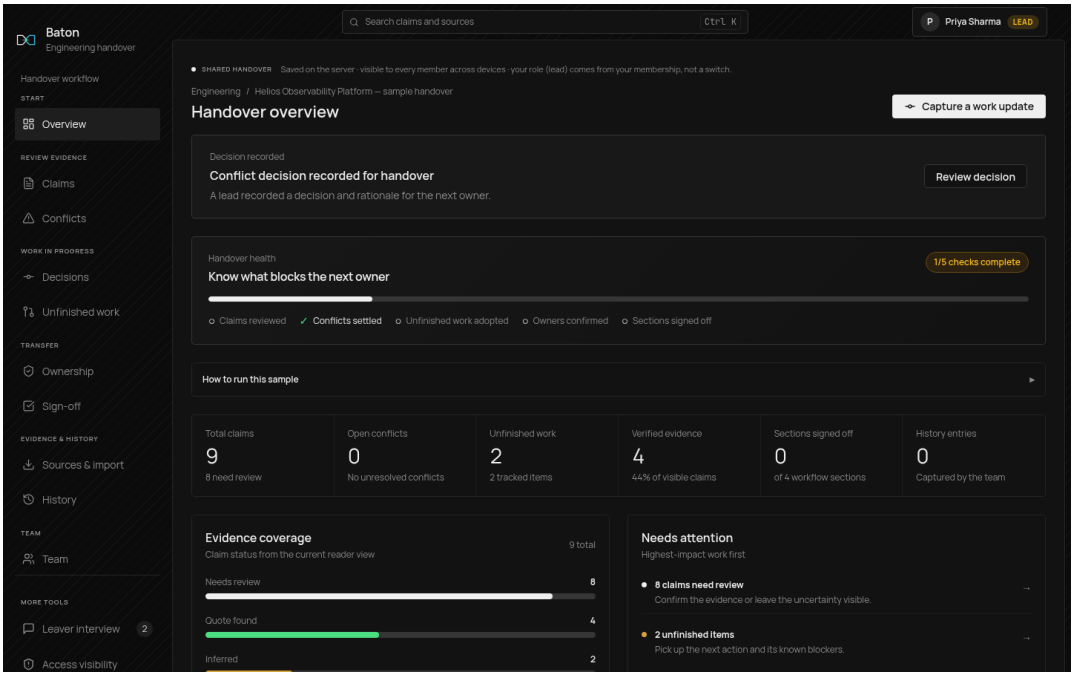


Figure 2 - The workspace overview: server-persisted handover state, clearance-filtered attention cards, and the member identity chip.

401
unauthenticated API calls, and demo-seat takeover attempts

403
role violations, e.g. successor calling lead-only settlement

404
non-member handover ids - no enumeration, no 403 leaks

SECTION 05

Improvement 2: Evidence-Backed GitHub Import

A handover ledger is only as credible as its sources. The import engine pulls real pull requests and commits from any public GitHub repository (or private one, with the importer's own OAuth token) through GitHub REST v3. Merged PRs are mapped to claims with deterministic verbatim excerpts; open PRs are triaged into unfinished work items with severity and suggested owners. Nothing in the pipeline calls an LLM - the mapper is a pure function covered by unit tests, so the same repository always yields the same claims.

Authentication for GitHub calls resolves through a three-tier chain: the importing user's OAuth token (5,000 requests/hour), a server-side PAT fallback, or unauthenticated public mode (60 requests/hour). The tier actually used is recorded on every import job and shown in the Sources panel, so rate-limit behaviour is visible rather than mysterious. Failures are first-class: a rate-limited or missing repository produces a friendly error with reset time, a failed job row stays visible in the panel, and unsupported source types - Slack, Linear, Jira, GitHub issues - are explicitly declared as unsupported in the UI instead of pretending to sync.

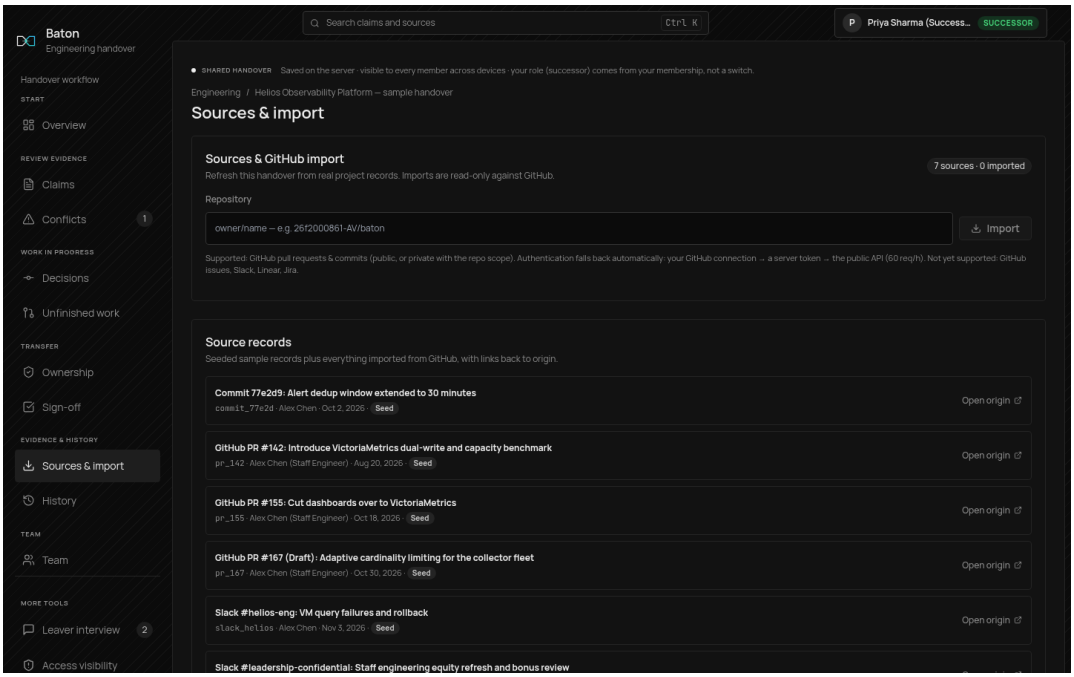


Figure 3 - The Sources panel: import jobs with per-job auth tier, origin links back to GitHub, and declared-unsupported source types.

3 tiers
user OAuth, server PAT, public - recorded per import job

5,000/h
user-token rate ceiling vs 60/h public fallback

0 LLMs
pure-function mappers; identical repo, identical claims

SECTION 06

Improvement 3: Shared, Verifiable Work History

Round 2's decision capture lived in one browser tab. Round 3 makes the history shared infrastructure: every work update is persisted with its author taken from the authenticated session - never free text - together with the rationale, the next action, and an optional link to an imported pull request. Settlements, imports, and membership changes flow into the same activity timeline, so the handover accumulates an audit trail across devices and users.

The design rule that matters most here is separation: the machine's verdict is rendered separately from the human's words. A quote check runs against the source text as a deterministic substring match with CRLF normalization and exact character offsets - a verified entry reads "quote found verbatim ... (chars 391-446)" with the offsets computed by the verifier, not typed by the author. A judge can therefore see at a glance which statements carry machine-checked evidence and which are a person's explanation, and the two are never visually conflated.

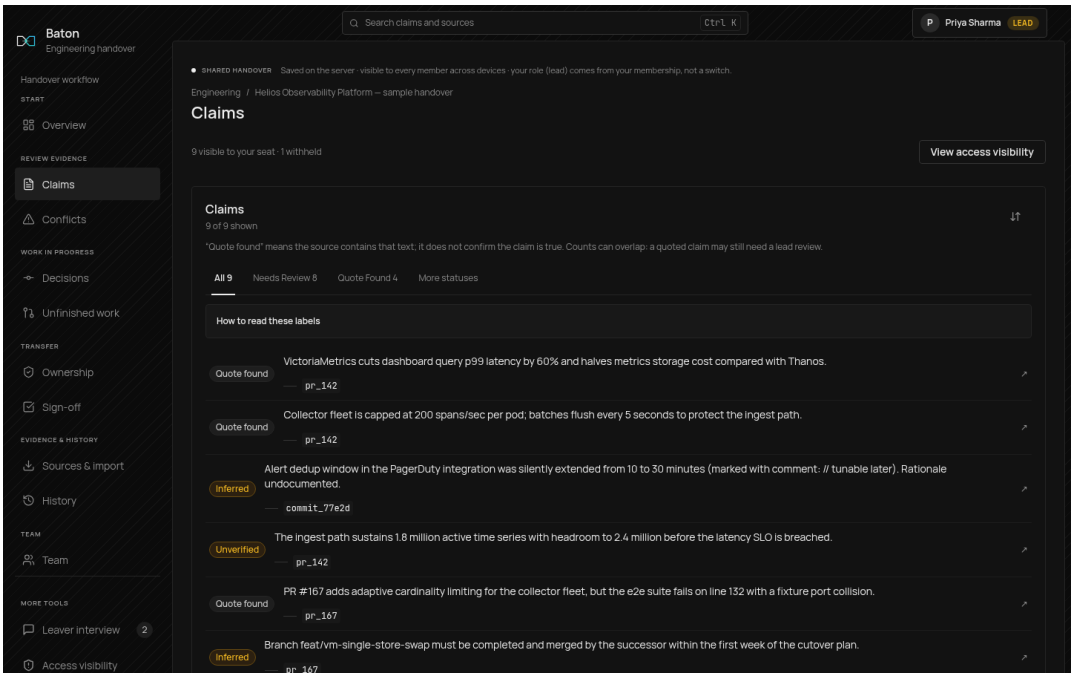


Figure 4 - Claims view: verification status per claim, disclosure chevrons, and the per-seat visibility counter after the Round 3 hardening pass.

Cross-device behaviour is the proof: settle a contradiction or capture an update in one browser, sign in as another member from a different device, and the entry is already there - attributed, timestamped, and linked to its evidence. The seed corpus ships with 11 quotes, 10 verbatim-consistent and one intentional paraphrase that correctly reports as unverified, so the badge cannot be faked by seed data.

SECTION 07

Architecture

Baton is a Next.js 14 App Router application served on Vercel, with React 18 and TypeScript throughout, Tailwind CSS for styling, Prisma v6 over Neon serverless PostgreSQL in production (SQLite for local development), and GitHub REST v3 as the only external API. The trust boundary is drawn once, in one place: every mutation passes the guard chain before it touches the database.

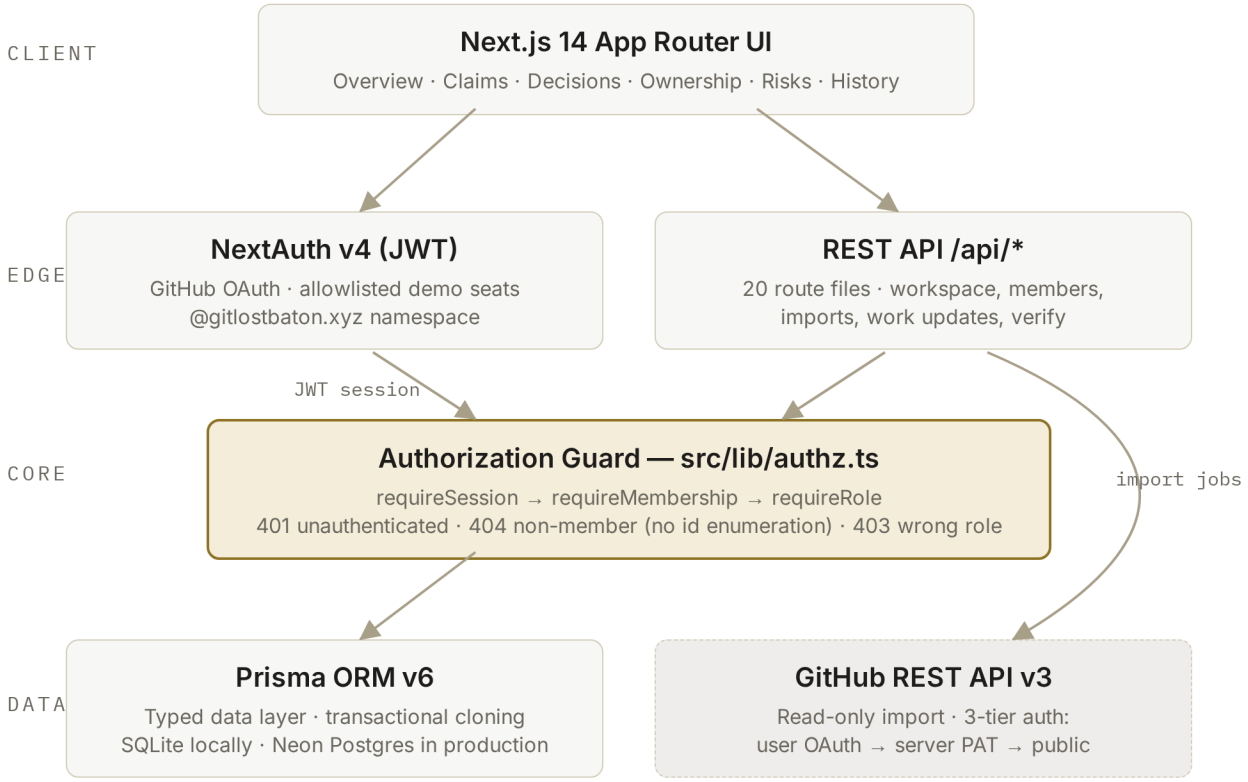


Figure 5 - Request path: the UI reaches the database only through the authorization guard; GitHub imports enter through the same guarded routes.

Two properties are deliberate. The client is untrusted - sessions carry only the user id and GitHub login, and every capability check re-reads the Membership row server-side. And the evidence path is deterministic end to end: the verifier is a substring computation with exact character offsets, the import mappers are pure functions, and no generative model sits anywhere in the chain. A tool that summarizes with an LLM can hallucinate a handover; Baton cannot, because nothing in its evidence path is generative.

SECTION 08

Technical Challenges and How They Were Handled

Four problems consumed most of the engineering judgment in this round. Each is stated here with its handling and the verification, because a challenge section that cannot be checked is marketing.

Challenge	Handling	Verified by
One schema, two databases: teams must run locally without provisioning Postgres, while production runs on Neon.	A provider-agnostic Prisma schema (sqlite mirror for development) with helper scripts that prefer bun and fall back to npx, so the documented npm-only quick start actually works on a fresh clone.	Fresh clone, npm install, db push, dev - the README path executed end to end.
Preventing optimistic UI drift: handover state is edited by several roles at once, and a stale client must never silently win.	All mutations POST to guarded routes and the UI renders the server's response; there are no optimistic local-only writes. Repeat actions and refreshes converge on server state.	Duplicate submits, mid-flight refresh, and a second device all re-hydrate from the server payload.
GitHub rate limits and failure modes are user-visible facts, not edge cases.	Per-job auth-tier recording, friendly rate-limit errors that include the reset time, failed job rows that persist in the Sources panel, and unsupported source types declared in the UI.	Unit tests for repo-input parsing and mappers; the rate-limited path exercised against the live API.
Judge ergonomics versus account safety: demo seats must be effortless, yet must never become a doorway into a real user's account.	Allowlisted demo logins resolving via the reserved @gitlostbaton.xyz e-mail namespace; demo provisioning happens on demand instead of hardcoding a production row id; debug logging is env-gated; lead-less handovers gain a founder-governance escape hatch so governance cannot dead-lock.	The takeover probe (signing in as a real GitHub username via the demo provider) returns 401 on production; 12/12 scripted probes pass.

The hardening pass behind the last row was driven by a self-audit loop: fifteen findings across severity levels, each fixed with a regression test or a scripted probe where feasible, and all gates (typecheck, lint, 19 unit tests, production build) re-run green before the fix commits landed on main.

SECTION 09

Round-wise Recap: Plan to Product

The original Round 1 proposal was broader than the build: PostgreSQL with pgvector, Slack and GitHub integrations, and LLM-assisted entailment. Round 2 deliberately narrowed to a self-contained demo and said so in its form. Round 3 converted the surviving core into a real product and shipped the three named improvements. The table below is the honest ledger of what moved and what did not.

Capability	Round 1 plan	Round 2 demo	Round 3 product
Persistence and accounts	Postgres + pgvector store	Browser localStorage, no accounts	NextAuth + Neon Postgres, multi-tenant, cross-device
Evidence intake	GitHub, Slack, Linear connectors	Fixed synthetic corpus	Live GitHub PR and commit import with origin links
Claim verification	LLM entailment scoring	Deterministic quote-presence check	Character-offset verifier, machine badge separate from author text
Access control	Production identity and roles	Persona dropdown, explicitly not a boundary	Server-side session-membership-role guards (401/404/403)
Deferred on purpose	-	Declared in the Round 2 form	pgvector search, Slack/Linear sync, LLM-assisted drafts - still future work, declared in the roadmap

The through-line is a product decision, not a limitation: Baton's value depends on its evidence being trustworthy, so generative components were pushed out of the critical path every round. What changed in Round 3 is that the boring, verifiable core - accounts, storage, imports, history - became real infrastructure instead of a simulation of one.

SECTION 10

Testing and Quality

Quality gates run on every commit and were re-run against the final submission tree on 14 November 2026; the production probes below were executed against gitlostbaton.xyz the same day, using the real CSRF sign-in flow a browser performs.

19/19
unit tests: verifier, GitHub mappers, authorization rules

12/12
scripted production probes passed on 14 Nov 2026

0 / 0
TypeScript errors / ESLint warnings on the final tree

Gate	Result	What it covers
tsc --noEmit	0 errors	Full-codebase strict typecheck
ESLint (next lint)	0 warnings, 0 errors	Components and API handlers
npm test	19/19 pass	Verifier: exact substring extraction, CRLF normalization, whitespace degradation, boundary and empty-quote rejection. Mappers: repo-input parsing (owner/name, URLs, .git), PR and commit mapping, deleted-author resilience, open-PR triage. Authorization: role gates, CUID id-shape behind 404 non-enumeration, clearance mapping, founder governance.
Seed integrity check	11/11 consistent	Every seeded quote verbatim against the corpus; claim_04 is an intentional paraphrase that correctly reports unverified
Production build	Passes	next build on the final commit
Live guards on production	401 / 403 / 404 / 200	Unauthenticated API calls 401; demo seat signs in and reaches its handover 200; foreign handover id 404; successor calling lead-only settle 403; takeover login rejected 401 with no session issued

Beyond the gates, the main journeys were walked end to end: the successor path (sign in, review decisions, adopt a work item, ask a question, accept a section, undo it) and the lead path (settle a contradiction with a note, inspect the exposure diff, import a repository). Empty states are designed rather than accidental - a fresh handover shows "Nothing has happened on this handover yet", a filtered claims view explains "No claims match this view" - and modals trap focus and close on Escape.

SECTION 11

Team, Disclosure, and Roadmap

Baton was built by a two-person team inside the event window. Aditya Verma (team leader) owned the architecture, the full-stack Next.js implementation, the Prisma schema, the NextAuth and GitHub integrations, and deployment. Manas Kapoor owned the documentation, the testing protocol, the user-journey specifications, and this submission dossier.

AI tool disclosure: AI coding assistants (Gemini, Antigravity CLI, and an independent audit agent for the hardening loop) were used for boilerplate acceleration, test drafting, and refactoring, under full human review. No generative model participates in the product's evidence path - the verifier and import mappers are deterministic code.

Horizon	Focus	Planned capabilities
Q1	Connectors	Slack app capturing architecture discussions from threads; Linear and Jira two-way sync turning unfinished work into sprint tickets
Q2	Attestation	Cryptographically signed handover certificates on full acceptance; dependency knowledge graph across decisions, assets, and owners
Q3	Identity	SCIM 2.0 with Okta, Google Workspace, and Entra ID so offboarding in HR triggers handover provisioning automatically
Q4	Compliance	SOC 2 Type II evidence export, tenant-level retention policies, and legal hold

The submitted artifact set: the live product at **gitlostbaton.xyz** (one-click demo seats on the sign-in page), the source repository at **github.com/26f2000861-AV/baton** (clone, npm install, and the README quick start reproduces the local build), and this dossier. Every claim in these pages was re-verified against the deployed product on 14 November 2026.